

WEEK 3 Assignment

Use Case

Build an **agentic content-curation system** that helps a user generate **high-quality LinkedIn post content** on a given topic.

Example user query:

“I want to post about recent trends in GenAI agents for backend engineers.”

The system must:

- discover **recent, relevant content**
- synthesize it into a **LinkedIn-style post**
- generate a **relevant image**
- expose the entire flow through a **planner-driven agentic system**

High-Level Architecture

The system must be **agent-oriented**, not a simple pipeline.

Agents

1. **Planner Agent**
2. **Executor Agent(s)** (tools)
3. **Generator Agent**
4. **Editor Agent**

Tech Constraints

- Use **LangChain** or **LlamaIndex** (developer chooses)
- Use **FastAPI** for backend agent orchestration

- Use **Streamlit** as UI (client only)
- Do **NOT** post to LinkedIn ,final output is shown in streamlit UI

Agent Responsibilities

1. Planner Agent (Core Brain)

Role

- Analyze user intent
- Decide **which tools to call**
- Decide **execution order**:
 - parallel vs sequential

Planner must output a plan like:

```
{  
  "steps": [  
    {"step": 1, "tool": "google_search_api", "depends_on": []},  
    {"step": 2, "tool": "x_search_api", "depends_on": []},  
    {"step": 3, "tool": "content_generator", "depends_on": [1, 2]},  
    {"step": 4, "tool": "content_editor", "depends_on": [3]},  
    {"step": 5, "tool": "image_generator", "depends_on": []}  
  ]  
}
```

How does this plan execute?

- **Steps 1, 2, and 5** can run in parallel (no dependencies).
- **Step 3** waits for search results from steps 1 and 2.
- **Step 4** waits for the generated content from step 3.
- The planner doesn't say "parallel" or "sequential" — **dependencies define execution order**.

2. Executor Agents / Tools

The planner can invoke these tools:

- **Search Tool**
 - Google Search API / Gemini / Perplexity (any free option)
 - Fetch latest content on the topic
- **Summarization Tool**
 - Condense raw articles into key points
- **Image Generation Tool**
 - Generate a visual matching the topic

Each tool must:

- have a clear schema
- return structured output
- be callable independently

3. Generator Agent

Role

- Take aggregated content
- Generate a **LinkedIn-style post**
 - concise
 - professional
 - audience-aware

Output must include:

- headline
- post body
- hashtags

4. Editor Agent

Role

- Improve clarity, tone, and structure
- Ensure post is suitable for LinkedIn
- Finalize the content

Execution Flow

1. User enters topic in **Streamlit**
2. Request goes to **FastAPI**
3. Planner Agent:
 - decides tools
 - defines parallel vs sequential steps
4. Executor Agents:

- run searches and summarization in parallel
- 5. Generator Agent:
 - creates the post
- 6. Editor Agent:
 - refines content
- 7. Image Generator:
 - generates image
- 8. Streamlit displays:
 - final post
 - generated image
 - planner execution steps

UI Requirements (Streamlit)

The UI must show:

- user input box
- final LinkedIn post text
- generated image
- **debug panel** with:
 - planner plan
 - tools executed
 - execution order (parallel / sequential)

Deliverables

1. Working code in a single GitHub repo
2. Planner output schema + execution logic
3. FastAPI endpoints:
 - `/plan`
 - `/execute`
4. Streamlit UI
5. Short README explaining:
 - agent roles
 - why planner is needed
 - one failure case you observed